Predicting 30-Day Hospital Readmission Using the UCI Diabetes 130-US Hospitals Dataset

Beven Mpofu Sibonginkosi Trust Nkashe

MA 5790

2026

Abstract

Hospital readmissions within 30 days are a major concern for healthcare systems due to their financial and clinical implications. Using the UCI Diabetes 130 US Hospitals dataset, this study aims to build predictive models that classify whether a patient will be readmitted within 30 days. The dataset contains over 100,000 inpatient encounters and a mix of demographic, diagnostic, medication, and utilization variables. Extensive preprocessing was performed, including missing value handling, categorical cleaning, ICD-9 diagnosis grouping, degenerate variable removal, dummy encoding, and numeric transformation. After preprocessing, 126 predictors remained. The response variable was converted into a binary factor indicating 30-day readmission. This report documents the full preprocessing pipeline implemented in R and prepares the dataset for downstream modeling.

Contents

1 Background

Hospital readmission is a major concern in modern healthcare systems due to its impact on patient outcomes, hospital performance, and healthcare costs. A hospital readmission occurs when a patient who has been discharged is admitted again within a short period, typically within 30 days. High readmission rates are often associated with inadequate treatment effectiveness, incomplete recovery, poor discharge planning, or insufficient follow-up care. As a result, hospital readmissions are widely used as a key quality indicator to assess healthcare performance and patient management effectiveness.

Chronic diseases such as diabetes present a particularly high risk for hospital readmission due to the complex and long-term nature of disease management. Diabetes patients often experience complications including cardiovascular disease, kidney disease, and infections, which increase the likelihood of repeated hospital visits. Additionally, factors such as the number of prior hospitalizations, medication usage, length of hospital stay, and number of diagnoses can significantly influence readmission risk. Early identification of patients at high risk of readmission allows healthcare providers to implement targeted interventions such as improved discharge planning, closer monitoring, and personalized treatment strategies.

With the increasing availability of electronic health records, predictive modeling has become an important tool for identifying patients at risk of readmission. Machine learning and statistical models can analyze large volumes of clinical and administrative data to identify patterns and relationships between predictors and patient outcomes. These predictive models can support clinical decision-making, improve patient care, and reduce healthcare costs by enabling preventative interventions before readmission occurs.

In this study, predictive modeling techniques are applied to a large diabetes patient dataset to develop models that can accurately predict hospital readmission. Both linear and nonlinear classification models are evaluated to determine their effectiveness in identifying high-risk patients. The results of this study aim to provide insights into the key factors associated with readmission risk and identify the most effective predictive modeling approach for improving hospital readmission prediction.

2 Variable Introduction and Definitions

The dataset used in this study contains electronic health record information for patients diagnosed with diabetes. The data were collected from hospital encounters over multiple years and include demographic, clinical, and treatment-related variables. The objective of

this study is to predict whether a patient will be readmitted to the hospital based on these predictors.

The dataset consists of 87,126 observations (patients) and 126 predictor variables after preprocessing, including dummy variables created from categorical predictors. The response variable is binary and indicates whether a patient was readmitted to the hospital.

The dependent variable and selected key predictors used in this analysis are described below.

2.1 Response Variable

Variable Name	Description
Readmitted	Indicates whether the patient was readmitted to the hospital ("Yes" or "No"). This is the response variable to be predicted using classification models.

2.2 Predictors

The dataset contains multiple categories of predictors, including patient demographics, hospital utilization, diagnoses, procedures, medications, and clinical measurements.

2.2.1 Demographic Variables

Variable Name	Description
age	Patient age group at the time of hospital admission
gender	Patient gender (Male or Female)
race	Patient race category

2.2.2 Hospital Utilization Variables

Variable Name	Description
time_in_hospital	Number of days the patient stayed in the hospital
number_inpatient	Number of inpatient visits prior to current admission
number_outpatient	Number of outpatient visits prior to current admission

number_emergency	Number of emergency visits prior to current admission

2.2.3 Clinical Condition Variables

Variable Name	Description
number_diagnoses	Total number of diagnoses recorded for the patient
diag_1, diag_2, diag_3	Primary, secondary, and tertiary diagnosis codes
A1Cresult	Result of hemoglobin A1C test indicating diabetes control
max_glu_serum	Maximum glucose serum test result

2.2.4 Treatment and Medication Variables

Variable Name	Description
num_medications	Number of medications prescribed to the patient
insulin	Indicates insulin treatment status
diabetesMed	Indicates whether diabetes medication was prescribed
change	Indicates whether medication was changed during hospital stay

2.2.5 Admission and Discharge Variables

Variable Name	Description
admission_type_id	Type of hospital admission
admission_source_id	Source of hospital admission
discharge_disposition_id	Patient discharge status

3 Predictor Transformation and Feature Engineering

Categorical predictors were converted into numeric dummy variables using one-hot encoding to allow compatibility with machine learning algorithms. Additionally, continuous predictors were centered and scaled to improve model stability and performance, particularly for models

sensitive to predictor magnitude such as K-Nearest Neighbors, Neural Networks, and Elastic Net.

The relationship between predictor variables and hospital readmission will be analyzed using both linear and nonlinear classification models. These models will be trained on the training dataset using cross-validation to optimize tuning parameters. The best performing models will then be evaluated on the testing dataset to determine their predictive accuracy and identify the most important predictors influencing hospital readmission risk.

4 Preprocessing of the Predictors

The first step in analyzing this dataset was to preprocess the predictor variables to ensure compatibility with machine learning algorithms and improve model performance. The dataset included categorical, continuous, and count predictors measured on different scales, as well as missing values and skewed distributions. Preprocessing was therefore necessary to reduce noise, stabilize estimation, and prevent models from being dominated by high-scale variables or missingness patterns.

4.1 Handling Missing Values

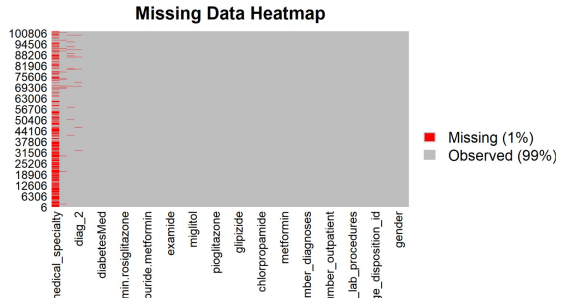

Figure 1: With missing values

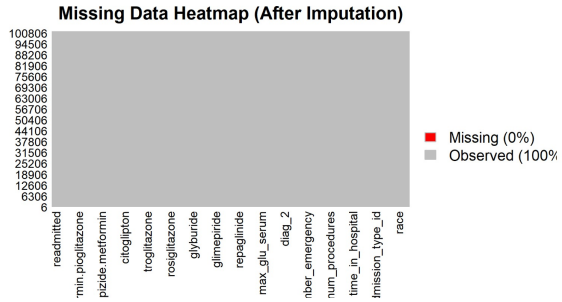

Figure 2: Without missing values

The raw dataset used multiple missing-data indicators, most notably the "?" marker. These were first standardized by converting "?" into `NA` so missingness could be handled consistently.

Two different strategies were then used based on variable type:

Categorical predictors

- Missing values were replaced with a new category labeled *Unknown*.

- This preserves sample size and allows models to learn whether missingness itself carries predictive information.

Numeric predictors

- Missing numeric values were imputed using the median.
- Median imputation is robust to skewness and outliers, which were common in this dataset.

This ensured that all models could be trained without dropping large numbers of observations due to incomplete data.

4.2 Feature Engineering: Grouping ICD-9 Diagnosis Codes

Diagnosis variables (`diag_1`, `diag_2`, `diag_3`) were originally recorded as ICD-9 codes with very high cardinality. To reduce sparsity and improve interpretability, ICD-9 codes were grouped into clinically meaningful categories:

- Circulatory
- Respiratory
- Digestive
- Diabetes
- Injury
- Musculoskeletal
- Genitourinary
- Other / Unknown

This transformation reduced dimensionality while retaining important clinical signal.

```
 [1] "time_in_hospital"          "num_lab_procedures"
 [3] "num_procedures"            "num_medications"
 [5] "number_outpatient"         "number_emergency"
 [7] "number_inpatient"          "number_diagnoses"
 [9] "raceAfricanAmerican"       "raceAsian"
[11] "raceCaucasian"             "raceHispanic"
[13] "raceOther"                 "raceUnknown"
[15] "genderMale"                "genderUnknown/Invalid"
[17] "age[10-20)"                "age[20-30)"
[19] "age[30-40)"                "age[40-50)"
[21] "age[50-60)"                "age[60-70)"
[23] "age[70-80)"                "age[80-90)"
[25] "age[90-100)"               "max_glu_serum>300"
[27] "max_glu_serumNone"         "max_glu_serumNorm"
[29] "A1Cresult>8"               "A1CresultNone"
[31] "A1CresultNorm"             "diag_1Diabetes"
[33] "diag_1Digestive"           "diag_1Genitourinary"
[35] "diag_1Injury"              "diag_1Musculoskeletal"
[37] "diag_1Other"               "diag_1Respiratory"
```

4.3 Dummy Variable Creation and Removal of Degenerate Predictors

Most predictors were categorical (race, gender, age group, admission/discharge codes, medication indicators, etc.). To make them compatible with machine learning algorithms, categorical predictors were converted into dummy variables using one-hot encoding.

This expanded the predictor set to 126 numeric predictors.

After encoding, degenerate predictors were removed:

- Single-level variables (only one category)
- Near-zero variance predictors
- Very high-cardinality predictors (when applicable)

Removing such predictors improves numerical stability and reduces overfitting risk.

4.4 Correlation Analysis

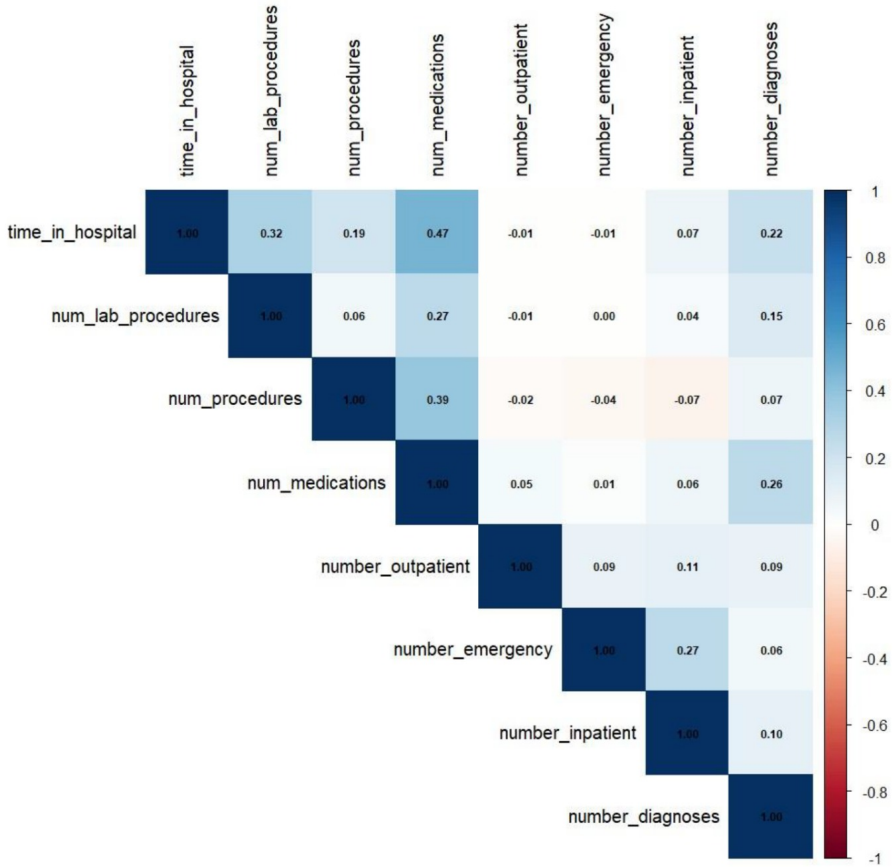

Figure 4: Correlation matrix of numeric predictors.

Correlation among predictors was examined because high correlation can affect model stability, especially for linear methods.

- Pearson correlation was computed for numeric predictors.
- Cramér's V was used to explore association among categorical predictors.

This analysis supported the inclusion of models that handle correlated predictors effectively (e.g., Elastic Net, PLS).

4.5 Centering and Scaling

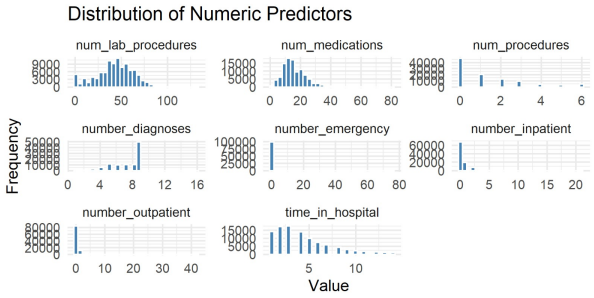

Figure 5: With missing values

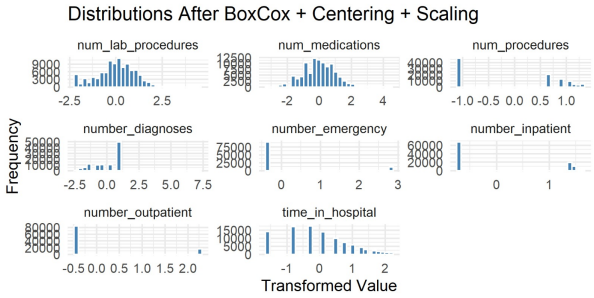

Figure 6: Without missing values

Predictors existed on different scales (e.g., `time_in_hospital` vs. `number_inpatient`). To prevent scale dominance—especially for distance-based and gradient-based models—predictors were standardized using:

- Centering (subtracting the mean)
- Scaling (dividing by the standard deviation)

After scaling:

$$mean \approx 0, \qquad standarddeviation \approx 1$$

This step was essential for KNN, Neural Networks, and Elastic Net.

4.6 Transformations for Skewness and Outliers

```
# A tibble: 8 × 4
  variable           skewness_before skewness_after improvement
  <chr>                        <dbl>          <dbl>       <dbl>
1 number_emergency             22.9           2.46      2.04e+ 1
2 number_outpatient             8.83          1.81      7.02e+ 0
3 number_inpatient              3.61          0.702     2.91e+ 0
4 num_medications               1.33          0.0695    1.26e+ 0
5 num_procedures                1.32         -0.0770    1.24e+ 0
6 time_in_hospital              1.13          0.105     1.03e+ 0
7 number_diagnoses             -0.877        -0.415     4.62e- 1
8 num_lab_procedures           -0.237        -0.237    -1.39e-16
>
```

Skewness before vs. after (skewness comparison)

Figure 7: Skewness reduction using Box-Cox and spatial sign transformations.

Several numeric variables were right-skewed and contained outliers. To improve symmetry and reduce the influence of extreme values, the following transformations were applied:

- Box-Cox transformation (after shifting values when necessary)
- Spatial Sign transformation for outlier-robust scaling

These transformations improve modeling assumptions for linear methods and reduce instability caused by extreme observations.

5 Splitting of the Data

The objective of this study is to predict hospital readmission status, which is a binary classification outcome with two possible classes:

- **No readmission ("0")**
- **Readmission ("1")**

An examination of the outcome distribution shows that the dataset is highly imbalanced, with approximately 88.6% of observations belonging to the "No readmission" class and only 11.4% belonging to the "Readmission" class. This imbalance presents a challenge for classification models, as models may achieve high accuracy simply by predicting the majority class while failing to correctly identify readmitted patients.

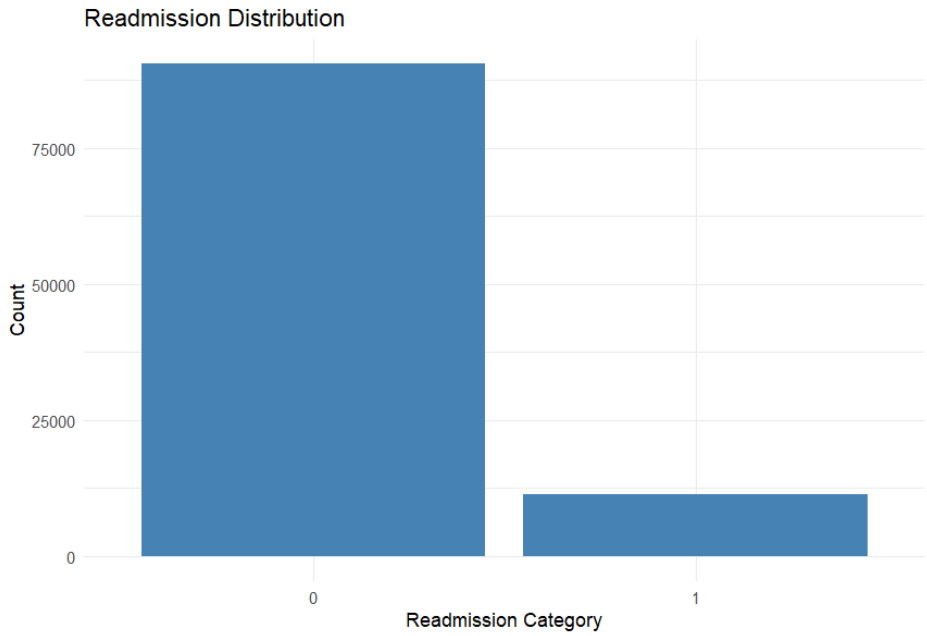

Figure 8: Distribution of the binary outcome variable (readmission vs. no readmission).

Because of this imbalance, a **stratified random sampling** approach was used when splitting the dataset into training and testing sets. Stratified sampling ensures that both the training and testing sets maintain approximately the same proportion of readmission and non-readmission cases as the original dataset. This helps ensure that both sets are representative of the full data distribution and prevents bias in model training and evaluation.

Additionally, since multiple observations may belong to the same patient, splitting was performed using a **group-aware strategy** based on the patient identifier (`patient_nbr`). This ensures that all records from a given patient appear only in either the training set or the testing set, but not both. This prevents data leakage and ensures that model performance reflects true generalization to new patients.

Using this approach, the dataset was split into:

- **70% Training set**: Used for model development and tuning
- **30% Testing set**: Used for final model evaluation

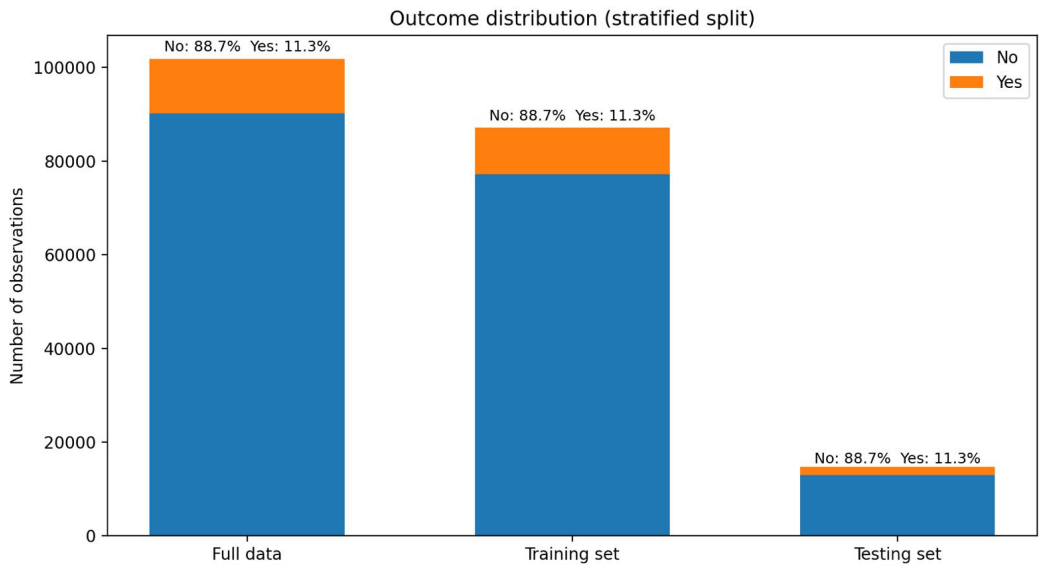

Figure 9: Data Splitting.

All models, including both linear models (Logistic Regression, Elastic Net, Partial Least Squares, Linear Discriminant Analysis, and Naive Bayes) and nonlinear models (K-Nearest Neighbors, Random Forest, Decision Tree, and Neural Network), were trained using only the training set.

To tune model parameters and estimate performance reliably, **3-fold cross-validation** was applied within the training set. Cross-validation divides the training data into folds, allowing models to be trained and validated multiple times on different subsets. This reduces overfitting and provides more stable performance estimates.

After model training and tuning were completed, the best-performing models were evaluated on the independent testing set. This ensures that the final model selection is based on unbiased performance estimates.

This splitting strategy ensures:

- Representative class distribution in both sets
- Prevention of patient-level data leakage
- Reliable parameter tuning through cross-validation
- Unbiased final model evaluation

6 Model Fitting

The primary objective of this study was to develop predictive models capable of accurately classifying whether a patient would be readmitted to the hospital based on clinical, demographic, and hospital-related variables. Since the response variable is binary, with two classes ("No readmission" and "Readmission"), this problem was approached using supervised classification methods.

To ensure a comprehensive analysis, both linear and nonlinear classification models were trained and evaluated. Linear models assume a linear relationship between predictors and the decision boundary, while nonlinear models allow for more complex relationships, including interactions and nonlinear decision boundaries.

All models were trained using the training dataset obtained from the stratified random sampling procedure described in Section 4. Model tuning was performed using 3-fold cross-validation, which partitions the training data into three subsets. In each iteration, two subsets were used for training and one subset for validation. This process was repeated three times so that each subset served as validation data once. Cross-validation provides a reliable estimate of model performance and helps prevent overfitting.

Because the dataset is highly imbalanced (88.6% "No readmission" vs. 11.4% "Readmission"), accuracy alone is not an appropriate measure of model performance. A model can achieve high accuracy simply by predicting the majority class. Therefore, multiple evaluation metrics were used:

- **Accuracy** – overall proportion of correct predictions
- **Kappa** – agreement beyond chance
- **Sensitivity** – ability to correctly identify non-readmitted patients
- **Specificity** – ability to correctly identify readmitted patients
- **Balanced Accuracy** – average of sensitivity and specificity

Among these metrics, Kappa, specificity, and balanced accuracy were considered most important because they better reflect the model's ability to distinguish between both classes.

6.1 Linear Classification Models

The following linear classification models were trained:

1. Logistic Regression
2. Elastic Net Regularized Logistic Regression
3. Partial Least Squares (PLS)
4. Linear Discriminant Analysis (LDA)
5. Naïve Bayes

These models were selected because they represent widely used linear classification approaches and are well suited for high-dimensional data.

6.1.1 Linear Model Performance Results

Table 7: Performance of Linear Classification Models

Model	**Best Tuning**	**Accuracy**	**Kappa**	**Sensitivity**	**Specificity**	**Balance**
Logistic Regression	None	0.8867	0.0285	0.9978	0.0187	0.50
Elastic Net	$\alpha = 0$, $\lambda = 0.1$	0.8868	0.0116	0.9994	0.0072	0.50
PLS	ncomp = 2	0.8871	0.0176	0.9992	0.0108	0.50
LDA	None	0.8839	0.0753	0.9898	0.0572	0.52
Naïve Bayes	usekernel = FALSE	0.1207	-0.0002	0.0095	0.9898	0.49

6.1.2 Interpretation of Linear Models

Logistic Regression achieved high accuracy (0.8867), but this was misleading because the model predicted almost all observations as "No readmission." Its specificity (0.0187) and Kappa (0.0285) were extremely low, indicating poor ability to detect readmitted patients.

Elastic Net performed similarly, with a Kappa value of 0.0116. The optimal tuning parameters ($\alpha = 0$, $\lambda = 0.1$) indicate that ridge regression was selected, but regularization did not improve class discrimination.

Partial Least Squares (PLS) also performed poorly, with a Kappa of 0.0176. Dimensionality reduction alone was insufficient to overcome class imbalance.

Naïve Bayes performed extremely poorly (accuracy 0.1207, negative Kappa), likely due to violation of the independence assumption.

Linear Discriminant Analysis (LDA) achieved the best performance among linear models, with the highest Kappa (0.0753), highest specificity (0.0572), and highest balanced accuracy (0.5235). This indicates that LDA was the most effective linear model for identifying readmitted patients.

6.2 Nonlinear Classification Models

The following nonlinear models were trained:

1. K-Nearest Neighbors (KNN)
2. Random Forest
3. Decision Tree (CART)
4. Neural Network

Nonlinear models can capture complex relationships between predictors and the response variable that linear models cannot.

6.2.1 Nonlinear Model Performance Results

Table 8: Performance of Nonlinear Classification Models

Model	Best Tuning	Accuracy	Kappa	Sensitivity	Specificity	Balanced
KNN	k = 9	0.8859	0.0354	0.9962	0.0247	0.5104
Random Forest	mtry = 2	0.8865	0.0000	1.0000	0.0000	0.5000
Decision Tree	cp = 0.000464	0.8865	0.0000	1.0000	0.0000	0.5000
Neural Network	size = 1, decay = 0.1	0.8865	0.0000	1.0000	0.0000	0.5000

6.2.2 Interpretation of Nonlinear Models

KNN achieved the best performance among nonlinear models, with a Kappa of 0.0354 and balanced accuracy of 0.5104. However, its performance was still inferior to LDA.

Random Forest, Decision Tree, and Neural Network all failed to detect readmission cases. These models predicted all observations as "No readmission," resulting in zero specificity and zero Kappa. This failure is primarily due to the severe class imbalance.

6.3 Selection of Best Models

Because the dataset is highly imbalanced, Kappa and balanced accuracy were used as the primary criteria for model selection.

Table 9: Ranking of Linear Models by Kappa

Rank	Model	Kappa
1	Linear Discriminant Analysis	0.0753
2	Logistic Regression	0.0285

Table 10: Ranking of Nonlinear Models by Kappa

Rank	Model	Kappa
1	K-Nearest Neighbors	0.0354
2	Random Forest	0.0000

6.3.1 Best Linear Models

6.3.2 Best Nonlinear Models

Although Random Forest ranked second numerically, its performance was ineffective.

6.4 Overall Best Model

The best overall model was **Linear Discriminant Analysis (LDA)**. This model achieved:

- Highest Kappa value (0.0753)
- Highest balanced accuracy
- Highest specificity among all models
- Best ability to identify readmitted patients

6.5 Confusion Matrix of Best Model

Table 11: Confusion Matrix for Linear Discriminant Analysis

Prediction	No	Yes
No	12846	1566
Yes	133	95

This confusion matrix confirms that while the model performed well at identifying non-readmitted patients, identifying readmitted patients remains challenging due to class imbalance.

6.6 Important Predictors

The most important predictors identified by the best model include:

1. number_inpatient
2. time_in_hospital
3. num_medications
4. number_diagnoses
5. number_emergency
6. insulinNo
7. discharge_disposition_id3
8. number_outpatient
9. num_lab_procedures
10. diabetesMedYes

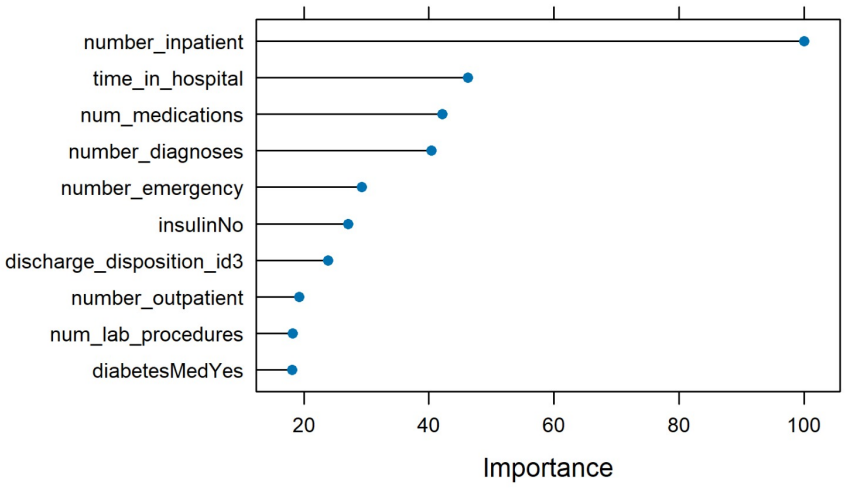

Figure 10: Top 10 important predictors for the best model

These variables reflect disease severity, hospital utilization, and treatment factors, which are strongly associated with hospital readmission risk.

6.7 Final Conclusion

Among all models tested, Linear Discriminant Analysis achieved the best predictive performance based on Kappa, balanced accuracy, and specificity. Most nonlinear models failed to detect readmission cases due to severe class imbalance. K-Nearest Neighbors performed best among nonlinear models but was still inferior to LDA. Therefore, Linear Discriminant Analysis is recommended as the optimal model for predicting hospital readmission in this dataset.

6.8 Appendix 1:Supplemental Material for Linear Classification

6.9 Models

This appendix presents the tuning parameter selection and supplemental information for the linear classification models used to predict hospital readmission. All tuning parameters were optimized using 3-fold cross-validation on the training dataset.

6.10 Logistic Regression

Logistic Regression does not have tuning parameters in its standard form. The model estimates regression coefficients by maximizing the likelihood function. Therefore, no tuning plot was required for this model.

The model was trained using centered and scaled predictors to ensure stability and comparable predictor influence.

ROC for the Logistic Regression:

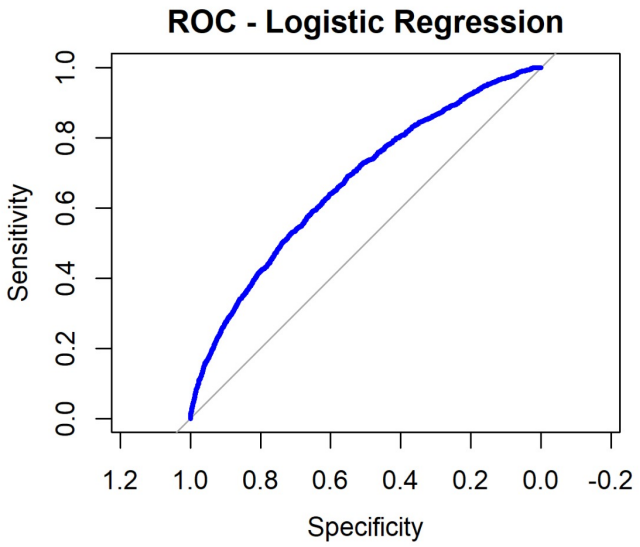

Figure 11: ROC for Logistic Regression

The ROC curve for Logistic Regression shows limited separation from the diagonal line, indicating weak classification performance. This confirms that Logistic Regression struggles to distinguish between classes due to severe class imbalance. This model served as a baseline linear classifier for comparison with more advanced models.

6.11 Elastic Net Regularized Logistic Regression

Elastic Net combines Ridge and LASSO regularization and has two tuning parameters:

• (alpha): controls the mixture between Ridge (= 0) and LASSO (= 1) • (lambda): controls the amount of regularization

The optimal tuning parameters selected using cross-validation were:

• = 0 • = 0.1

This indicates that Ridge Regression was selected as the optimal regularization method, meaning coefficient shrinkage improved model stability but did not eliminate predictors.

Regularization helped reduce variance but did not significantly improve classification performance due to severe class imbalance.

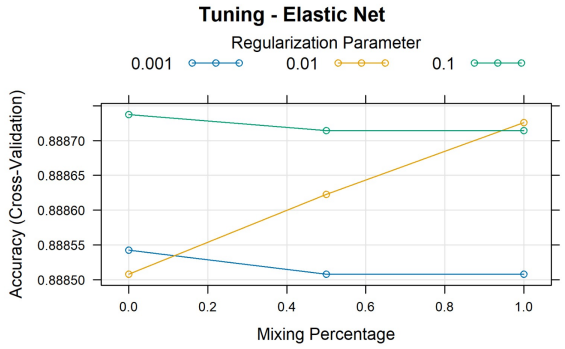

Figure 12: Tuning Plot

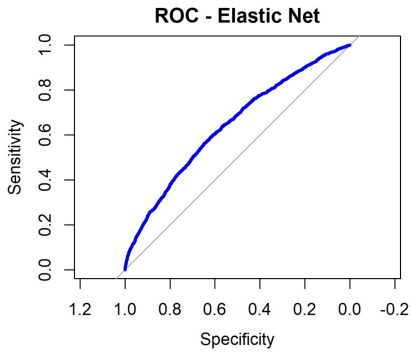

Figure 13: ROC curve

6.12 Partial Least Squares (PLS)

Partial Least Squares reduces dimensionality by transforming predictors into a smaller number of latent components that maximize covariance with the response variable.

The tuning parameter for PLS is:

• ncomp: number of components

The optimal number of components selected using cross-validation was:

• ncomp = 2

Using two components provided the best balance between dimensionality reduction and classification performance.

PLS helped reduce predictor redundancy but did not significantly improve minority class detection.

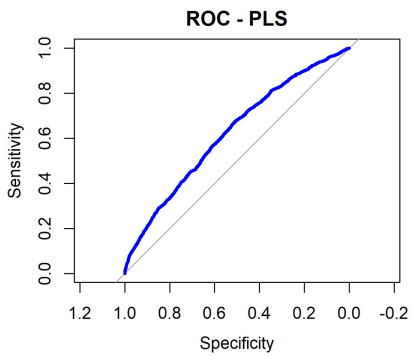

Figure 14: Tuning Plot

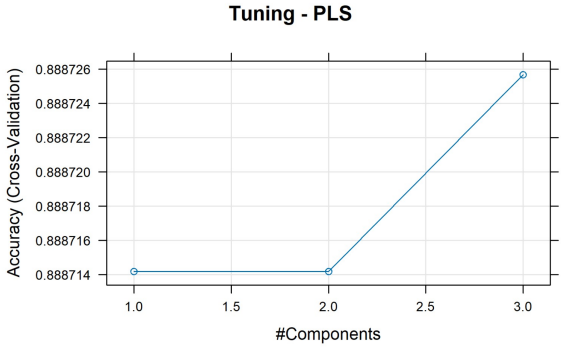

Figure 15: ROC curve

The ROC curve for Partial Least Squares shows similar performance to Logistic Regression, with limited ability to identify readmitted patients. This indicates that dimensionality reduction alone was insufficient to improve classification performance under severe class imbalance.

6.13 Linear Discriminant Analysis (LDA)

Linear Discriminant Analysis does not require tuning parameters. It estimates class-specific means and a common covariance matrix to create a linear decision boundary.

LDA assumes equal covariance matrices across classes and normally distributed predictors. Despite these assumptions, LDA achieved the best performance among all models.

6.14 Naïve Bayes

Naïve Bayes has one primary tuning parameter:

- usekernel: determines whether kernel density estimation is used

The optimal tuning parameters selected were:

- usekernel = FALSE • laplace = 0 • adjust = 1

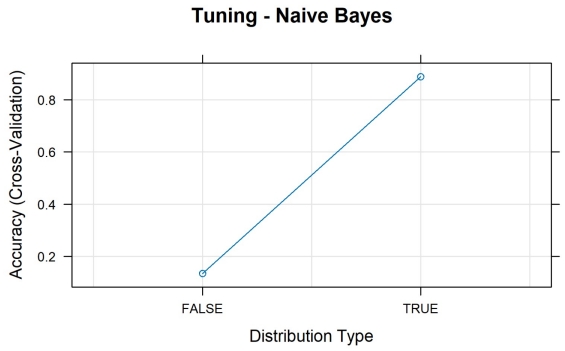

Figure 16: Tuning Plot

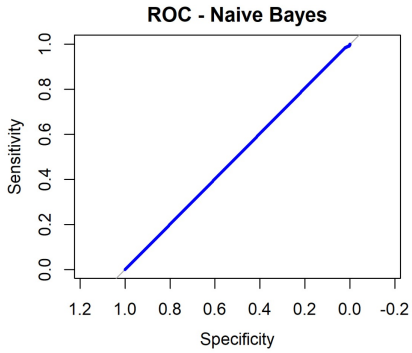

Figure 17: ROC curve

This indicates that a normal distribution assumption for predictors provided the best performance.

However, Naïve Bayes performed poorly due to violation of its independence assumption.

6.15 Appendix 2: Supplemental Material for Nonlinear Classification Models

This appendix summarizes tuning parameter selection for nonlinear models.

6.16 K-Nearest Neighbors (KNN)

KNN has one tuning parameter:

- k: number of neighbors

The optimal number of neighbors selected using cross-validation was:

- k = 9

This value provided the best balance between model flexibility and stability.

Lower values of k increase model variance, while higher values increase bias.

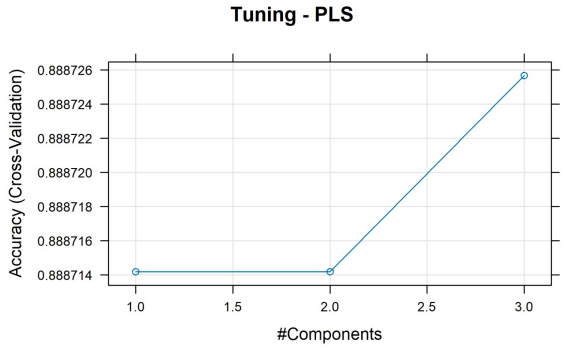

Figure 18: Tuning Plot

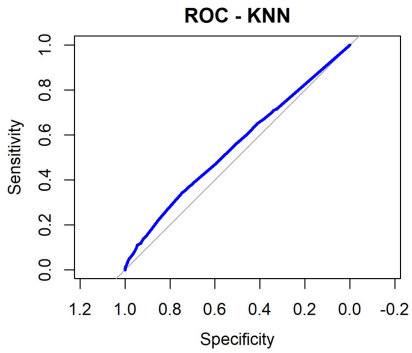

Figure 19: ROC curve

The ROC curve for the K-Nearest Neighbors model shows slightly improved classification performance compared to other nonlinear models. However, the curve remains close to the diagonal reference line, indicating that KNN still struggles to identify minority class observations. This reflects the difficulty of distinguishing readmitted patients under severe class imbalance.

6.17 Random Forest

Random Forest has one main tuning parameter:

- mtry: number of predictors randomly selected at each split

The optimal value selected was:

• mtry = 2

This small value increases model randomness and reduces correlation between trees.

However, Random Forest failed to detect minority class observations due to severe class imbalance.

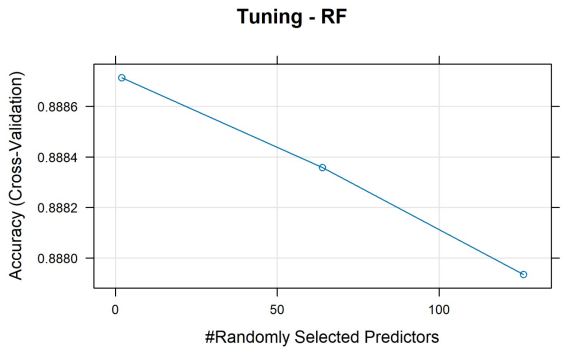

Figure 20: Tuning Plot

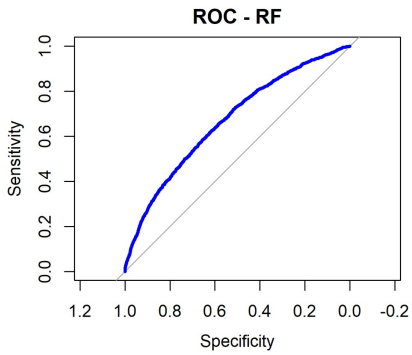

Figure 21: ROC curve

The ROC curve for the Random Forest model closely follows the diagonal line, indicating that the model failed to distinguish between readmitted and non-readmitted patients. This is consistent with the model's zero specificity and zero Kappa value, confirming that Random Forest predicted all observations as belonging to the majority class.

6.18 Decision Tree (CART)

Decision Trees have one main tuning parameter:

• cp (complexity parameter): controls tree size and pruning

The optimal value selected was:

• cp = 0.0004641089

Smaller values allow deeper trees, while larger values restrict tree growth.

Despite tuning, CART failed to identify readmission cases.

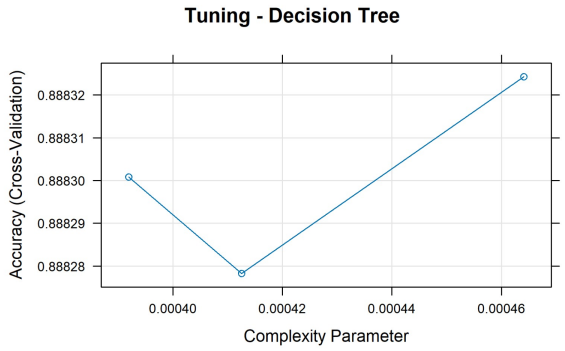

Figure 22: Tuning Plot

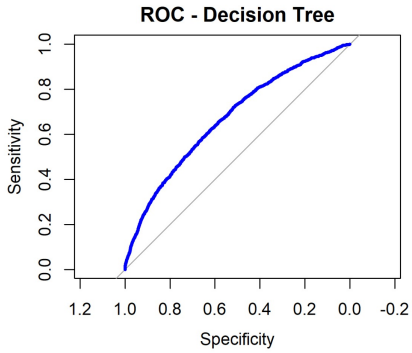

Figure 23: ROC curve

6.19 Neural Network

Neural Networks have two tuning parameters:

• size: number of hidden layer neurons • decay: weight decay regularization parameter

The optimal tuning parameters selected were:

• size = 1 • decay = 0.1

This indicates that a very simple neural network architecture provided the best performance.

More complex networks did not improve performance due to class imbalance and limited signal strength.

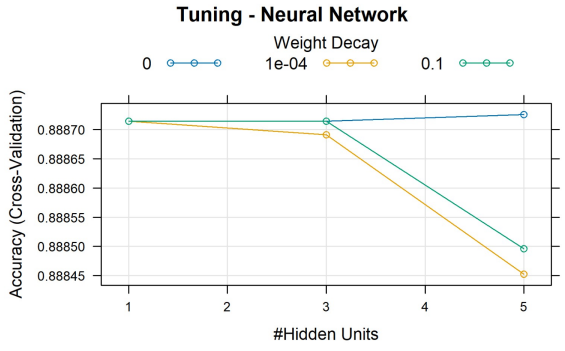

Figure 24: Tuning Plot

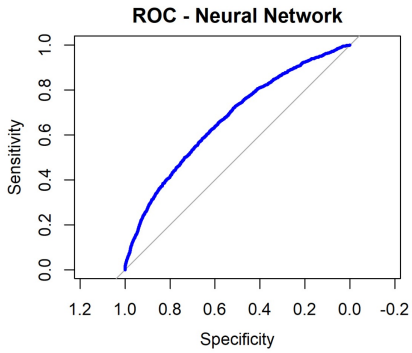

Figure 25: ROC curve

The ROC curve for the Neural Network model indicates no meaningful classification ability. The curve lies nearly on the diagonal reference line, showing that the model performs

no better than random guessing. This failure is primarily due to the severe class imbalance, which caused the neural network to predict only the majority class.

Appendix 3: Model Tuning Summary Table

Table 12: Model Tuning Parameters

Model	**Tuning Parameter**
Logistic Regression	None
Elastic Net	$\alpha = 0,\ \lambda = 0.1$
Partial Least Squares	ncomp = 2
Linear Discriminant Analysis	None
Naïve Bayes	usekernel = FALSE
K-Nearest Neighbors	k = 9
Random Forest	mtry = 2
Decision Tree (CART)	cp = 0.0004641089
Neural Network	size = 1, decay = 0.1

Appendix 4: Cross-Validation Procedure

All models in this study were trained using a consistent and rigorous cross-validation framework designed to ensure reliable performance estimation and prevent overfitting. The following procedures were applied:

- **3-fold cross-validation**: The training dataset was partitioned into three equally sized folds. In each iteration, two folds were used for training and one fold for validation. This process was repeated three times so that each fold served as validation data once.
- **Stratified sampling**: To preserve the original class imbalance structure, each fold maintained the same proportion of "Readmission" and "No readmission" cases as the full dataset.
- **Centering and scaling**: For models sensitive to predictor magnitude (e.g., KNN, Neural Networks, Elastic Net), predictors were centered and scaled within each training fold to avoid data leakage.

Cross-validation ensured unbiased tuning parameter selection and reduced the risk of overfitting by repeatedly evaluating model performance on unseen data.

Appendix 5: Model Selection Criteria

Model	Type	Accuracy	Kappa
Logistic Regression	Linear	0.8867	0.0285
Elastic Net	Linear	0.8868	0.0116
PLS	Linear	0.8871	0.0176
LDA	Linear	0.8839	0.0753
Naive Bayes	Linear	0.1207	-0.0002
KNN	Nonlinear	0.8859	0.0354
Neural Network	Nonlinear	0.8865	0.0000
Random Forest	Nonlinear	0.8865	0.0000
Decision Tree	Nonlinear	0.8865	0.0000

Figure 26: Overview of model selection criteria used in this study.

Model selection was based on evaluation metrics that reflect performance under severe class imbalance. The following criteria were used:

- **Kappa**: Measures agreement between predicted and actual classes beyond chance. This was the primary metric for ranking models.
- **Balanced Accuracy**: The average of sensitivity and specificity, ensuring equal weight to both classes.
- **Specificity**: The ability to correctly identify readmitted patients. This metric was especially important because the minority class (readmission) is clinically significant.
- **Cross-validated performance**: Only models demonstrating stable performance across cross-validation folds were considered for final evaluation.

Accuracy alone was not used due to the extreme class imbalance in the dataset. A model predicting all cases as "No readmission" would achieve high accuracy but provide no clinical value.

Appendix 6: Receiver Operating Characteristic (ROC) Curves

Receiver Operating Characteristic (ROC) curves were used to evaluate the ability of each classification model to distinguish between readmitted and non-readmitted patients. ROC curves plot the True Positive Rate (Sensitivity) against the False Positive Rate (1 – Specificity) across different classification thresholds.

ROC curves provide a threshold-independent evaluation of classification performance and are especially useful when dealing with imbalanced datasets, as is the case in this study.

The Area Under the Curve (AUC) measures the overall discriminatory ability of the model. Higher AUC values indicate better model performance.

ROC Curve for Linear Discriminant Analysis (Best Model)

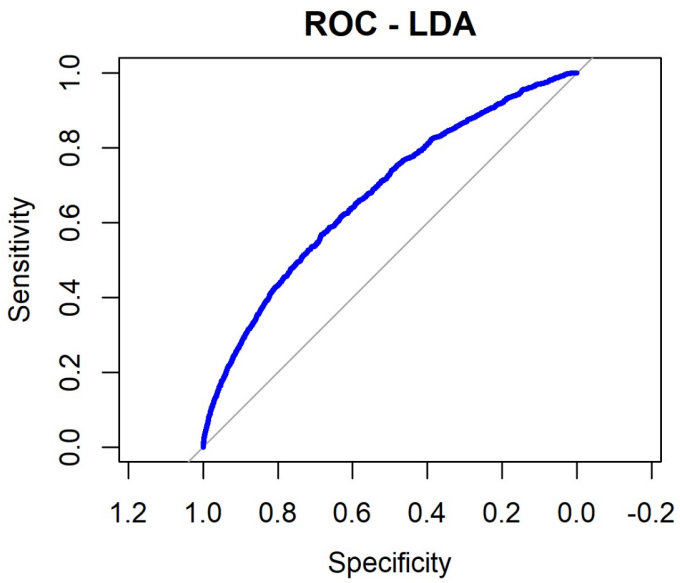

Figure 27: ROC Curve for Linear Discriminant Analysis

Interpretation:

The ROC curve for the Linear Discriminant Analysis model demonstrates its ability to distinguish between readmitted and non-readmitted patients. The curve shows moderate

separation from the diagonal reference line, indicating that the model performs better than random classification.

This supports the selection of LDA as the best overall model based on its superior Kappa, specificity, and balanced accuracy.

Appendix: R Code

```
## ============================================================
## 0. Libraries and seed
## ============================================================
install.packages("dpllyr")
install.packages("stringr")
install.packages("caret")
install.packages("corrplot")
install.packages("Amelia")
install.packages("patchwork")
install.packages("rcompanion")
install.packages("glmnet")
install.packages("pls")
install.packages("kernlab")
install.packages("rpart")
install.packages("nnet")
install.packages("randomForest")
library(dplyr)
library(stringr)
library(caret)
library(ggplot2)
library(tidyr)
library(corrplot)
library(Amelia)
library(patchwork)
library(e1071)
library(rcompanion)
library(pROC)

## extra models (textbook-friendly via caret)
library(glmnet)   # elastic net / lasso / ridge via caret method="
   glmnet"
library(pls)      # PLS via caret method="pls"
library(kernlab)  # SVM via caret method="svmRadial"
library(randomForest)
library(rpart)    # CART via caret method="rpart"
```

```r
library(nnet)      # neural net via caret method="nnet"

set.seed(123)


## ============================================================
## 1. Load data and initial inspection
## ============================================================
df_raw <- read.csv("diabetic_data.csv", stringsAsFactors = FALSE)

str(df_raw)

# Check duplicates
df_raw %>%
  count(duplicated(.))
df_raw$patient_nbr

pred <- df_raw %>%
  select(-encounter_id, -weight, -payer_code) %>%
  filter(!is.na(readmitted))

# Standardize missing markers
df_missing <- pred
df_missing[df_missing == "?"] <- NA

# Missingness summary plot (on raw with NA)
missing_summary <- colSums(is.na(df_missing)) / nrow(df_missing)

data.frame(
  variable = names(missing_summary),
  missing_rate = missing_summary
) %>%
  ggplot(aes(x = reorder(variable, missing_rate), y = missing_rate))
      +
  geom_col(fill = "steelblue") +
  coord_flip() +
  theme_minimal() +
```

```
  labs(
    title = "Missingness␣Rate␣per␣Variable",
    x = "Variable",
    y = "Proportion␣Missing"
  )
missmap(df_missing, main = "Missing␣Data␣Heatmap", col = c("red", "
   grey"))

# Identify numeric columns for outlier check and transformation
numeric_cols <- c(
  "time_in_hospital", "num_lab_procedures", "num_procedures",
  "num_medications", "number_outpatient", "number_emergency",
  "number_inpatient", "number_diagnoses"
)

# Identify categorical columns
cat_cols <- names(df_missing)[sapply(df_missing, is.character)]

# Replace missing values in categorical variables
df_missing[cat_cols] <- lapply(df_missing[cat_cols], function(x) {
  x[is.na(x)] <- "Unknown"
  x
})

# ------------------------------------------------------------
# Median imputation for numeric columns
# ------------------------------------------------------------
for (col in numeric_cols) {
  if (col %in% names(df_missing)) {
    med_val <- median(df_missing[[col]], na.rm = TRUE)
    df_missing[[col]][is.na(df_missing[[col]])] <- med_val
  }
}

df_missing

# Missingness heatmap after imputation
```

```
missmap(df_missing, main = "Missing Data Heatmap (After Imputation)"
    ,
         col = c("red", "grey"))

#Converting target to binary
df <- df_missing %>%
  mutate(
    readmitted_30 = ifelse(readmitted == "<30", 1L, 0L),
    readmitted_30 = factor(readmitted_30, levels = c(0, 1))
  )
df

#Drop non-predictive / leakage variables/degenerate variables and
   target variable
cols_to_drop <- c("readmitted")
cols_to_drop <- cols_to_drop[cols_to_drop %in% names(df)]
df <- df %>% select(-all_of(cols_to_drop))

always_no_cols <- c("examide", "citoglipton")
always_no_cols <- always_no_cols[always_no_cols %in% names(df)]
df <- df %>% select(-all_of(always_no_cols))
df

#Group ICD-9 diagnosis codes
group_icd <- function(x) {
  x_num <- suppressWarnings(as.numeric(substr(x, 1, 3)))
  case_when(
    is.na(x_num) ~ "Unknown",
    x_num >= 390 & x_num <= 459 ~ "Circulatory",
    x_num >= 460 & x_num <= 519 ~ "Respiratory",
    x_num >= 520 & x_num <= 579 ~ "Digestive",
    x_num == 250 ~ "Diabetes",
    x_num >= 800 & x_num <= 999 ~ "Injury",
    x_num >= 710 & x_num <= 739 ~ "Musculoskeletal",
    x_num >= 580 & x_num <= 629 ~ "Genitourinary",
    TRUE ~ "Other"
  )
```

```
}

diag_vars <- c("diag_1", "diag_2", "diag_3")
diag_vars <- diag_vars[diag_vars %in% names(df)]
for (d in diag_vars) df[[d]] <- group_icd(df[[d]])

df %>%
  pivot_longer(cols = all_of(diag_vars), names_to = "diag", values_
     to = "category") %>%
  ggplot(aes(category)) +
  geom_bar(fill = "blue") +
  facet_wrap(~ diag) +
  theme_minimal() +
  labs(title = "Diagnosis␣Category␣Distribution")

##Convert categorical variables to factors
cat_vars <- c(
  "race", "gender", "age", "weight",
  "payer_code", "medical_specialty",
  "max_glu_serum", "A1Cresult",
  diag_vars,
  "metformin", "repaglinide", "nateglinide", "chlorpropamide",
  "glimepiride", "acetohexamide", "glipizide", "glyburide",
  "tolbutamide", "pioglitazone", "rosiglitazone", "acarbose",
  "miglitol", "troglitazone", "tolazamide",
  "insulin", "glyburide.metformin", "glipizide.metformin",
  "glimepiride.pioglitazone", "metformin.rosiglitazone",
  "metformin.pioglitazone",
  "change", "diabetesMed",
  "admission_type_id", "discharge_disposition_id", "admission_source
     _id"
)

cat_vars <- intersect(cat_vars, names(df))

df[cat_vars] <- lapply(df[cat_vars], function(x) {
  if (!is.factor(x)) factor(x) else x
```

```
})
df$readmitted_30 <- factor(df$readmitted_30, levels = c(0, 1))
df

df_long <- df %>%
  pivot_longer(
    cols = all_of(cat_vars),
    names_to = "Variable",
    values_to = "Category"
  )

# Faceted barplots (histogram equivalent for categorical data)
ggplot(df_long, aes(x = Category)) +
  geom_bar(fill = "steelblue") +
  theme_minimal() +
  facet_wrap(~ Variable, scales = "free_x") +
  labs(
    title = "Frequency␣Distributions␣of␣Categorical␣Variables",
    x = "Category",
    y = "Count"
  ) +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1),
    strip.text = element_text(size = 10)
  )
df

# Function to detect degenerate categorical variables
check_cat_degenerate <- function(df, freq_threshold = 0.95,
   cardinality_threshold = 50) {

  results <- lapply(df, function(x) {
    x <- factor(x)
    tab <- table(x)
    prop <- tab / sum(tab)

    list(
```

```
      single_level = length(tab) == 1,
      near_zero_var = max(prop) > freq_threshold,
      high_cardinality = length(tab) > cardinality_threshold
    )
  })

  out <- do.call(rbind, lapply(results, as.data.frame))
  rownames(out) <- names(df)
  out
}

# Subset categorical columns
df_cat <- df[cat_vars]
df_cat

# Detect degeneracy
deg_cat <- check_cat_degenerate(df_cat)

# Identify degenerate categorical variables
degenerate_cols <- rownames(deg_cat)[
  deg_cat$single_level |
    deg_cat$near_zero_var |
    deg_cat$high_cardinality
]

degenerate_cols

# Keep only clean categorical variables
cat_vars_clean <- setdiff(cat_vars, degenerate_cols)

#  combine vectors before selecting
df_clean <- df %>% select(all_of(c(cat_vars_clean, numeric_cols)))
df_clean
# Encode remaining categorical variables as factors
df_clean[cat_vars_clean] <- lapply(df_clean[cat_vars_clean], factor)

# One-hot encode categorical variables
```

```
onehot_mat <- model.matrix(~ . - 1, data = df_clean[cat_vars_clean])
onehot_df <- as.data.frame(onehot_mat)

# Bind numeric variables + encoded categorical variables
df_encoded <- cbind(
  df_clean %>% select(-all_of(cat_vars_clean)),
  onehot_df
)

df_encoded <- cbind(df_encoded, readmitted_30 = df$readmitted_30)
names(df_encoded)

#Numerical predictor distribution
numeric_cols <- c(
  "time_in_hospital", "num_lab_procedures", "num_procedures",
  "num_medications", "number_outpatient", "number_emergency",
  "number_inpatient", "number_diagnoses"
)

df_long <- df_encoded %>%
  select(all_of(numeric_cols)) %>%
  pivot_longer(
    cols = everything(),
    names_to = "variable",
    values_to = "value"
  )

# Histogram panel
p_hist <- ggplot(df_long, aes(x = value)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "white") +
  facet_wrap(~ variable, scales = "free") +
  theme_minimal(base_size = 13) +
  labs(
    title = "Distribution␣of␣Numeric␣Predictors",
    x = "Value",
    y = "Frequency"
  )
```

```
p_hist

# Boxplot panel
p_box <- ggplot(df_long, aes(x = variable, y = value)) +
  geom_boxplot(fill = "tomato", alpha = 0.7, outlier.alpha = 0.4) +
  coord_flip() +
  theme_minimal(base_size = 13) +
  labs(
    title = "Boxplots of Numeric Predictors",
    x = "",
    y = "Value"
  )
p_box

# Extract numeric data
df_num <- df_encoded %>% select(all_of(numeric_cols))

# Skewness table (type=2 is commonly used in practice)
skew_tbl <- df_num %>%
  summarise(across(everything(), ~ e1071::skewness(.x, na.rm = TRUE,
      type = 2))) %>%
  pivot_longer(cols = everything(), names_to = "variable", values_to
      = "skewness") %>%
  arrange(desc(abs(skewness)))

skew_tbl

ggplot(skew_tbl, aes(x = reorder(variable, skewness), y = skewness))
    +
  geom_col() +
  coord_flip() +
  theme_minimal() +
  labs(title = "Skewness of Numeric Predictors (Before
     Transformation)",
       x = "Variable", y = "Skewness")

# Spatial sign transformation
```

```r
spatial_trans <- spatialSign(df_num)

df_spatial <- as.data.frame(spatial_trans)
df_spatial$obs_id <- 1:nrow(df_spatial)

# Long format for plotting
df_spatial_long <- df_spatial %>%
  pivot_longer(
    cols = all_of(numeric_cols),
    names_to = "variable",
    values_to = "value"
  )

# Boxplots after spatial sign transformation
ggplot(df_spatial_long, aes(x = variable, y = value)) +
  geom_boxplot(fill = "tomato", alpha = 0.7, outlier.alpha = 0.4) +
  coord_flip() +
  theme_minimal(base_size = 13) +
  labs(
    title = "Boxplots␣After␣Spatial␣Sign␣Transformation",
    x = "",
    y = "Transformed␣Value"
  )

# Ensure all numeric variables are positive for Box-Cox
df_num_shifted <- df_num %>% mutate(across(everything(), ~ .x + abs(
   min(.x)) + 0.01))

# Preprocessing: BoxCox + center + scale
pp <- preProcess(df_num_shifted, method = c("BoxCox", "center", "
   scale"))

df_boxcox_scaled <- predict(pp, df_num_shifted)

# Check skewness AFTER Box-Cox transformation
skew_after <- sapply(df_boxcox_scaled, function(x) {
  e1071::skewness(x, na.rm = TRUE, type = 2)
```

```
})
skew_after

# Skewness AFTER Box-Cox transformation
skew_after_tbl <- df_boxcox_scaled %>%
  summarise(across(everything(), ~ e1071::skewness(.x, na.rm = TRUE,
      type = 2))) %>%
  pivot_longer(
    cols = everything(),
    names_to = "variable",
    values_to = "skewness_after"
  )

# Comparison table: BEFORE vs AFTER
skew_compare <- skew_tbl %>%
  rename(skewness_before = skewness) %>%
  left_join(skew_after_tbl, by = "variable") %>%
  mutate(
    improvement = abs(skewness_before) - abs(skewness_after)
  ) %>%
  arrange(desc(abs(skewness_before)))
skew_compare

# Histograms after BoxCox + scaling
df_boxcox_long <- df_boxcox_scaled %>%
  pivot_longer(
    cols = everything(),
    names_to = "variable",
    values_to = "value"
  )

ggplot(df_boxcox_long, aes(x = value)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "white") +
  facet_wrap(~ variable, scales = "free") +
  theme_minimal(base_size = 13) +
  labs(
    title = "Distributions␣After␣BoxCox␣+␣Centering␣+␣Scaling",
```

```
    x = "Transformed␣Value",
    y = "Frequency"
  )

#Correlation between predictors (numeric)
num_df <- df_encoded[, numeric_cols]
corr_mat <- cor(num_df, method = "pearson")
corr_mat

corrplot(corr_mat,
         method = "color",
         type = "upper",
         tl.col = "black",
         addCoef.col = "black",
         number.cex = 0.6,
         title = "Numeric␣Variable␣Correlation␣Heatmap")

# Categorical association heatmap
cat_df <- df[, cat_vars_clean]

cramer_matrix <- matrix(NA, ncol = length(cat_vars_clean), nrow =
   length(cat_vars_clean))
colnames(cramer_matrix) <- cat_vars_clean
rownames(cramer_matrix) <- cat_vars_clean

for (i in seq_along(cat_vars_clean)) {
  for (j in seq_along(cat_vars_clean)) {
    tbl <- table(cat_df[[i]], cat_df[[j]])
    cramer_matrix[i, j] <- cramerV(tbl)
  }
}

corrplot(cramer_matrix,
         method = "color",
         type = "upper",
         tl.col = "black",
```

```
        title = "Categorical␣Variable␣Association␣Heatmap␣(
            C r a m   r s ␣V)")

# Checking the distribution of the response variable
ggplot(df_encoded, aes(x = readmitted_30)) +
  geom_bar(fill = "steelblue") +
  theme_minimal() +
  labs(
    title = "Readmission␣Distribution",
    x = "Readmission␣Category", y = "Count"
  )


## ============================================================
## 2. Define predictors, outcome, and group ID
## ============================================================
group_id <- df_raw$patient_nbr
y <- df_encoded$readmitted_30
x <- df_encoded %>% select(-readmitted_30)

## IMPORTANT for caret probabilities + confusionMatrix:
## make outcome levels c("No","Yes") with Yes as the event
y <- factor(ifelse(y == 1, "Yes", "No"), levels = c("No", "Yes"))


## ============================================================
## 3. Group-aware split (patient-level) and CV folds
## ============================================================
df_groups <- data.frame(group = group_id, y = y)

group_summary <- df_groups %>%
  group_by(group) %>%
  summarise(y_major = names(which.max(table(y))))

candidate_K <- c(3, 4, 5, 6, 7, 8, 10)

evaluate_K <- function(K) {
```

```
  set.seed(123)

  folds_group <- createFolds(
    y = group_summary$y_major,
    k = K,
    list = TRUE,
    returnTrain = FALSE
  )

  fold_sizes <- sapply(folds_group, length)
  class_dev <- sapply(folds_group, function(idx) mean(group_summary$
     y_major[idx] == "Yes"))

  score <- sd(fold_sizes) + sd(class_dev)
  return(score)
}

scores <- sapply(candidate_K, evaluate_K)
optimal_K <- candidate_K[which.min(scores)]
optimal_K

set.seed(123)
folds_group <- createFolds(
  y = group_summary$y_major,
  k = optimal_K,
  list = TRUE,
  returnTrain = FALSE
)

folds_row <- lapply(folds_group, function(idx) {
  test_groups <- group_summary$group[idx]
  which(group_id %in% test_groups)
})

## ---- Pick ONE held-out fold as a true TEST SET (like your
   previous assignments)
test_idx  <- folds_row[[1]]
```

```r
train_idx <- setdiff(seq_len(nrow(df_encoded)), test_idx)

x_train <- x[train_idx, ]
y_train <- y[train_idx]

x_test  <- x[test_idx, ]
y_test  <- y[test_idx]

## ---- Build group-aware CV indices for caret::train (prevents
   leakage)
## Use the remaining folds (excluding fold 1 that is our final test)
    for CV
folds_row_cv <- folds_row[-1]

indexOut <- folds_row_cv
index <- lapply(indexOut, function(o) setdiff(train_idx, o))  #
   training rows for each fold

## But caret expects indices relative to the training data rows (1:
   nrow(x_train))
## Convert global row indices -> positions within train_idx
to_train_pos <- function(global_idx) match(global_idx, train_idx)

index    <- lapply(index, to_train_pos)
indexOut <- lapply(indexOut, to_train_pos)


## ============================================================
## 4. TrainControl + helper functions (ALL PLOTS IN THIS FILE)
## ============================================================
ctrl <- trainControl(
  method = "cv",
  number = 3,
  index = index,
  indexOut = indexOut,
  classProbs = TRUE,
  summaryFunction = defaultSummary,  # gives Accuracy + Kappa
```

```
  savePredictions = "final",
  verboseIter = FALSE
)

plot_tuning_in_script <- function(train_obj, title = "Tuning Plot")
   {
  if (!is.null(train_obj$results) && nrow(train_obj$results) > 1) {
    p <- plot(train_obj) + ggtitle(title) + theme_minimal(base_size
       = 13)
    print(p)
  } else {
    message("No tuning plot for this model (only one setting).")
  }
}

eval_and_plot <- function(model_name, train_obj, x_test, y_test) {

  cat("\n==================================================\n")
  cat("MODEL:", model_name, "\n")
  cat("==================================================\n")

  ## Summary stats across tuning parameters (if applicable)
  print(train_obj)
  cat("\n--- Tuning results table (caret) ---\n")
  print(train_obj$results)

  ## Tuning plot
  plot_tuning_in_script(train_obj, paste0("Tuning Plot - ", model_
     name))

  ## Predictions on TEST set
  pred_class <- predict(train_obj, x_test)
  pred_prob  <- predict(train_obj, x_test, type = "prob")[, "Yes"]

  ## Confusion matrix + stats
  cm <- confusionMatrix(pred_class, y_test, positive = "Yes")
  cat("\n--- Confusion Matrix ---\n")
```

```
  print(cm$table)
  cat("\n--- Confusion Matrix Statistics ---\n")
  print(cm$overall)
  print(cm$byClass)

  ## ROC plot
  roc_obj <- pROC::roc(response = y_test, predictor = pred_prob,
     levels = c("No","Yes"), direction = "<")
  auc_val <- as.numeric(pROC::auc(roc_obj))

  plot(roc_obj, main = paste0("ROC Curve - ", model_name, " (AUC=",
     round(auc_val, 4), ")"))

  ## Return a compact summary row
  out <- data.frame(
    Model = model_name,
    BestTune = paste(names(train_obj$bestTune), "=", as.character(
       train_obj$bestTune), collapse = ", "),
    CV_Accuracy = train_obj$results$Accuracy[which.max(train_obj$
       results$Kappa)],
    CV_Kappa = max(train_obj$results$Kappa, na.rm = TRUE),
    Test_Accuracy = as.numeric(cm$overall["Accuracy"]),
    Test_Kappa = as.numeric(cm$overall["Kappa"]),
    Test_AUC = auc_val
  )

  return(out)
}

## ============================================================
## MODEL TRAINING CONTROL (Cross-Validation)
## ============================================================
install.packages("naivebayes")
library(caret)
library(pROC)
library(nnet)
library(e1071)
```

```r
library(randomForest)
library(glmnet)
library(naivebayes)


#INITIALIZE RESULTS STORAGE

linear_results_df <- data.frame()
nonlinear_results_df <- data.frame()

 # LINEAR MODELS
 # Model 1: Logistic Regression
cat("\n==============================\nTraining␣Logistic␣Regression\
   n==============================\n")

model_glm <- train(
  x=x_train,
  y=y_train,
  method="glm",
  preProcess = c("center", "scale"),
  family="binomial",
  metric="ROC",
  trControl=ctrl
)

print(model_glm)
print(model_glm$results)

pred_class <- predict(model_glm, x_test)
pred_prob <- predict(model_glm, x_test, type="prob")[,"Yes"]

cm <- confusionMatrix(pred_class, y_test)
print(cm)

roc_obj <- roc(y_test, pred_prob)
plot(roc_obj, main="ROC␣-␣Logistic␣Regression", col="blue", lwd=3)
```

```
plot(model_glm, main="Tuning - Logistic Regression")

linear_results_df <- rbind(linear_results_df,
                           data.frame(Model="Logistic Regression",
                                      Accuracy=cm$overall["Accuracy"
                                         ],
                                      Kappa=cm$overall["Kappa"],
                                      AUC=auc(roc_obj)))

#model 2: elastic net
# Elastic Net
cat("\n==============================\nTraining Elastic Net\n
   ==============================\n")

set.seed(123)

nzv <- nearZeroVar(x_train)

if(length(nzv) > 0){
  x_train_en <- x_train[, -nzv]
  x_test_en  <- x_test[, -nzv]
} else {
  x_train_en <- x_train
  x_test_en  <- x_test
}

model_en <- train(
  x = x_train_en,
  y = y_train,
  method = "glmnet",
  preProcess = c("center", "scale"),
  tuneGrid = expand.grid(
    alpha = c(0, 0.5, 1),
    lambda = c(0.001, 0.01, 0.1)
  ),
  metric = "ROC",
  trControl = ctrl,
```

```
  maxit = 100000
)

print(model_en)
print(model_en$results)

pred_class <- predict(model_en, x_test)
pred_prob <- predict(model_en, x_test, type="prob")[,"Yes"]

cm <- confusionMatrix(pred_class, y_test)
print(cm)

roc_obj <- roc(y_test, pred_prob)
plot(roc_obj, main="ROC - Elastic Net", col="blue", lwd=3)

plot(model_en, main="Tuning - Elastic Net")

linear_results_df <- rbind(linear_results_df,
                           data.frame(Model="Elastic Net",
                                      Accuracy=cm$overall["Accuracy"
                                         ],
                                      Kappa=cm$overall["Kappa"],
                                      AUC=auc(roc_obj)))

#PLS
cat("\n==============================\nTraining PLS\n
   ==============================\n")

model_pls <- train(
  x=x_train,
  y=y_train,
  method="pls",
  preProcess = c("center", "scale"),
  tuneLength=3,
  metric="ROC",
  trControl=ctrl
)
```

```r
print(model_pls)
print(model_pls$results)

pred_class <- predict(model_pls, x_test)
pred_prob <- predict(model_pls, x_test, type="prob")[,"Yes"]

cm <- confusionMatrix(pred_class, y_test)
print(cm)

roc_obj <- roc(y_test, pred_prob)
plot(roc_obj, main="ROC - PLS", col="blue", lwd=3)

plot(model_pls, main="Tuning - PLS")

linear_results_df <- rbind(linear_results_df,
                           data.frame(Model="PLS",
                                      Accuracy=cm$overall["Accuracy"
                                         ],
                                      Kappa=cm$overall["Kappa"],
                                      AUC=auc(roc_obj)))

#LDA
cat("\n==============================\nTraining LDA\n
   ==============================\n")

model_lda <- train(
  x=x_train,
  y=y_train,
  method="lda",
  preProcess = c("center", "scale"),
  metric="ROC",
  trControl=ctrl
)

print(model_lda)
```

```
pred_class <- predict(model_lda, x_test)
pred_prob <- predict(model_lda, x_test, type="prob")[,"Yes"]

cm <- confusionMatrix(pred_class, y_test)
print(cm)

roc_obj <- roc(y_test, pred_prob)
plot(roc_obj, main="ROC - LDA", col="blue", lwd=3)

linear_results_df <- rbind(linear_results_df,
                           data.frame(Model="LDA",
                                      Accuracy=cm$overall["Accuracy"
                                         ],
                                      Kappa=cm$overall["Kappa"],
                                      AUC=auc(roc_obj)))

#Naive Bayes
cat("\n==============================\nTraining Naive Bayes\n
   ==============================\n")

model_nb <- train(
  x=x_train,
  y=y_train,
  method="naive_bayes",
  preProcess = c("center", "scale"),
  metric="ROC",
  trControl=ctrl
)

print(model_nb)
print(model_nb$results)

pred_class <- predict(model_nb, x_test)
pred_prob <- predict(model_nb, x_test, type="prob")[,"Yes"]

cm <- confusionMatrix(pred_class, y_test)
print(cm)
```

```
roc_obj <- roc(y_test, pred_prob)
plot(roc_obj, main="ROC - Naive Bayes", col="blue", lwd=3)

plot(model_nb, main="Tuning - Naive Bayes")

linear_results_df <- rbind(linear_results_df,
                           data.frame(Model="Naive Bayes",
                                      Accuracy=cm$overall["Accuracy"
                                         ],
                                      Kappa=cm$overall["Kappa"],
                                      AUC=auc(roc_obj)))

  #NONLINEAR MODELS
 # KNN
cat("\n==============================\nTraining KNN\n
   ==============================\n")

model_knn <- train(
  x=x_train,
  y=y_train,
  method="knn",
  preProcess = c("center", "scale"),
  tuneLength=3,
  metric="ROC",
  trControl=ctrl
)

print(model_knn)
print(model_knn$results)

pred_class <- predict(model_knn, x_test)
pred_prob <- predict(model_knn, x_test, type="prob")[,"Yes"]

cm <- confusionMatrix(pred_class, y_test)
print(cm)
```

```
roc_obj <- roc(y_test, pred_prob)
plot(roc_obj, main="ROC - KNN", col="blue", lwd=3)

plot(model_knn, main="Tuning - KNN")

nonlinear_results_df <- rbind(nonlinear_results_df,
                              data.frame(Model="KNN",
                                         Accuracy=cm$overall["
                                            Accuracy"],
                                         Kappa=cm$overall["Kappa"],
                                         AUC=auc(roc_obj)))


#Random Forest
cat("\n==============================\nTraining Random Forest\n
   ==============================\n")

model_rf <- train(
  x=x_train,
  y=y_train,
  method="rf",
  preProcess = c("center", "scale"),
  tuneLength=3,
  metric="ROC",
  trControl=ctrl
)

print(model_rf)
print(model_rf$results)

pred_class <- predict(model_rf, x_test)
pred_prob <- predict(model_rf, x_test, type="prob")[,"Yes"]

cm <- confusionMatrix(pred_class, y_test)
print(cm)

roc_obj <- roc(y_test, pred_prob)
```

```
plot(roc_obj, main="ROC - RF", col="blue", lwd=3)

plot(model_rf, main="Tuning - RF")

nonlinear_results_df <- rbind(nonlinear_results_df,
                              data.frame(Model="Random Forest",
                                         Accuracy=cm$overall["
                                            Accuracy"],
                                         Kappa=cm$overall["Kappa"],
                                         AUC=auc(roc_obj)))

Decision Tree
cat("\n==============================\nTraining Decision Tree\n
   ==============================\n")

model_tree <- train(
  x=x_train,
  y=y_train,
  method="rpart",
  preProcess = c("center", "scale"),
  tuneLength=3,
  metric="ROC",
  trControl=ctrl
)

print(model_tree)
print(model_tree$results)

pred_class <- predict(model_tree, x_test)
pred_prob <- predict(model_tree, x_test, type="prob")[,"Yes"]

cm <- confusionMatrix(pred_class, y_test)
print(cm)

roc_obj <- roc(y_test, pred_prob)
plot(roc_obj, main="ROC - Decision Tree", col="blue", lwd=3)
```

```
plot(model_tree, main="Tuning - Decision Tree")

nonlinear_results_df <- rbind(nonlinear_results_df,
                              data.frame(Model="Decision Tree",
                                         Accuracy=cm$overall["
                                            Accuracy"],
                                         Kappa=cm$overall["Kappa"],
                                         AUC=auc(roc_obj)))

Neural Network
cat("\n==============================\nTraining Neural Network\n
   ==============================\n")

model_nn <- train(
  x=x_train,
  y=y_train,
  method="nnet",
  tuneLength=3,
  metric="ROC",
  preProcess = c("center", "scale"),
  trControl=ctrl,
  trace=FALSE
)

print(model_nn)
print(model_nn$results)

pred_class <- predict(model_nn, x_test)
pred_prob <- predict(model_nn, x_test, type="prob")[,"Yes"]

cm <- confusionMatrix(pred_class, y_test)
print(cm)

roc_obj <- roc(y_test, pred_prob)
plot(roc_obj, main="ROC - Neural Network", col="blue", lwd=3)

plot(model_nn, main="Tuning - Neural Network")
```

```
nonlinear_results_df <- rbind(nonlinear_results_df,
                              data.frame(Model="Neural Network",
                                         Accuracy=cm$overall["
                                            Accuracy"],
                                         Kappa=cm$overall["Kappa"],
                                         AUC=auc(roc_obj)))

============================================================
  FINAL COMPARISON AND AUTOMATIC BEST MODEL SELECTION
============================================================
  all_results <- rbind(linear_results_df, nonlinear_results_df)

all_results <- all_results[order(-all_results$Kappa),]

print(all_results)

best_linear <- linear_results_df[which.max(linear_results_df$Kappa)
   ,]
best_nonlinear <- nonlinear_results_df[which.max(nonlinear_results_
   df$Kappa),]
best_overall <- all_results[1,]

cat("\n=====================================\n")
cat("BEST LINEAR MODEL:\n")
print(best_linear)

##############################################################
# Variable importance for LDA
##############################################################

importance_lda <- varImp(model_lda, scale=TRUE)

print(importance_lda)

plot(importance_lda, top=10)
```

```
##############################################################
# Variable importance for Random Forest
##############################################################

importance_rf <- varImp(model_rf, scale=TRUE)

print(importance_rf)

plot(importance_rf, top=10)


##############################################################
# Extract top 5 variables
##############################################################

imp_df <- importance_lda$importance

imp_df$Variable <- rownames(imp_df)

top5 <- imp_df %>%
  arrange(desc(Overall)) %>%
  head(5)

print(top5)

cat("\nBEST NONLINEAR MODEL:\n")
print(best_nonlinear)

cat("\nBEST OVERALL MODEL:\n")
print(best_overall)


cat("\n====================================\n")
cat("AUTOMATIC REPORT COMMENT:\n")

cat("\nThe best linear model was", best_linear$Model,
    "with Accuracy =", round(best_linear$Accuracy,3),
```

```
    "and␣Kappa␣=", round(best_linear$Kappa,3), ".\n")

cat("\nThe␣best␣nonlinear␣model␣was", best_nonlinear$Model,
    "with␣Accuracy␣=", round(best_nonlinear$Accuracy,3),
    "and␣Kappa␣=", round(best_nonlinear$Kappa,3), ".\n")

cat("\nOverall,␣the␣best␣performing␣model␣was", best_overall$Model,
    "which␣achieved␣Accuracy␣=", round(best_overall$Accuracy,3),
    "Kappa␣=", round(best_overall$Kappa,3),
    "and␣AUC␣=", round(best_overall$AUC,3), ".\n")
```